```python
import random
import math
import numpy as np


CITY_COORDINATES = {
    0: (0, 0),    # Starting Depot
    1: (2, 5),    # City 1
    2: (5, 8),    # City 2
    3: (8, 3),    # City 3
    4: (4, 1)     # City 4
}

NUM_CITIES = len(CITY_COORDINATES)

DIMENSION = NUM_CITIES - 1

def calculate_distance(c1, c2):
    return math.sqrt((c1[0] - c2[0])**2 + (c1[1] - c2[1])**2)


DISTANCE_MATRIX = np.zeros((NUM_CITIES, NUM_CITIES))
for i in range(NUM_CITIES):
    for j in range(NUM_CITIES):
        if i != j:
            DISTANCE_MATRIX[i][j] = calculate_distance(CITY_COORDINATES[i], CITY_COORDINATES[j])

def continuous_to_route(position):
    """Maps continuous numbers to a valid delivery sequence using Rank-Ordering."""

    sorted_indices = np.argsort(position)

    ordered_cities = [idx + 1 for idx in sorted_indices]

    return [0] + ordered_cities + [0]

def calculate_total_distance(position):
    """Calculates total trip distance for a given continuous position."""
    route = continuous_to_route(position)
```

```python
def calculate_total_distance(position):
    """Calculates total trip distance for a given continuous position."""
    route = continuous_to_route(position)
    total_dist = 0
    for i in range(len(route) - 1):
        total_dist += DISTANCE_MATRIX[route[i]][route[i+1]]
    return total_dist


class DeliveryCuckooSearch:
    def __init__(self, num_nests=20, iterations=60, pa=0.25, beta=1.5):
        self.num_nests = num_nests
        self.iterations = iterations
        self.pa = pa
        self.beta = beta
        self.lb, self.ub = -5.0, 5.0

        num = math.gamma(1 + self.beta) * math.sin(math.pi * self.beta / 2)
        den = math.gamma((1 + self.beta) / 2) * self.beta * (2 ** ((self.beta - 1) / 2))
        self.sigma = (num / den) ** (1 / self.beta)

    def _levy_flight(self, position, best_position):
        """Generates a random walk using Levy distribution."""
        u = np.random.normal(0, self.sigma, size=DIMENSION)
        v = np.random.normal(0, 1, size=DIMENSION)
        step = u / (np.abs(v) ** (1 / self.beta))

        step_size = 0.05 * step * (position - best_position)
        return np.clip(position + step_size, self.lb, self.ub)

    def run(self):

        nests = np.random.uniform(self.lb, self.ub, (self.num_nests, DIMENSION))
        fitness = np.array([calculate_total_distance(nest) for nest in nests])

        best_idx = np.argmin(fitness)
        best_nest = nests[best_idx].copy()
        best_fitness = fitness[best_idx]
```

```
        for t in range(self.iterations):

            i = random.randint(0, self.num_nests - 1)
            new_nest = self._levy_flight(nests[i], best_nest)
            new_fitness = calculate_total_distance(new_nest)

            j = random.randint(0, self.num_nests - 1)
            if new_fitness < fitness[j]:
                nests[j] = new_nest
                fitness[j] = new_fitness

            sort_idx = np.argsort(fitness)
            cutoff = int(self.num_nests * (1 - self.pa))

            for idx in sort_idx[cutoff:]:
                r1, r2 = random.choice(sort_idx), random.choice(sort_idx)
                step_size = random.random() * (nests[r1] - nests[r2])
                nests[idx] = np.clip(nests[idx] + step_size, self.lb, self.ub)
                fitness[idx] = calculate_total_distance(nests[idx])


            current_best_idx = np.argmin(fitness)
            if fitness[current_best_idx] < best_fitness:
                best_fitness = fitness[current_best_idx]
                best_nest = nests[current_best_idx].copy()

        return continuous_to_route(best_nest), best_fitness


def print_delivery_schedule(route, total_distance):
    print("\n" + "="*55)
    print(f"{'CUCKOO SEARCH OPTIMIZED ROUTE TIMELINE':^55}")
    print("="*55)
    print(f"{'Stop Order':<12} | {'Action / Destination Location':<40}")
    print("-"*55)

    for i, city in enumerate(route[:-1]):
        loc_name = "Starting Depot" if city == 0 else f"City {city}"
```

```
        print("="*55)
        print(f"{'Stop Order':<12} | {'Action / Destination Location':<40}")
        print("-"*55)

        for i, city in enumerate(route[:-1]):
            loc_name = "Starting Depot" if city == 0 else f"City {city}"
            print(f"Stop {i+1:<6} | Depart/Arrive at {loc_name}")

        print(f"Stop {len(route):<6} | Return safely back to Starting Depot")
        print("="*55)

        path_map = " -> ".join(["Depot" if c == 0 else f"City {c}" for c in route])
        print(f"Visual Route Map:\n{path_map}")
        print(f"\nMinimized Total Travel Distance: {total_distance:.2f} miles/km")
        print("="*55)

    if __name__ == "__main__":
        cs_optimizer = DeliveryCuckooSearch(num_nests=25, iterations=50)
        optimal_route, min_distance = cs_optimizer.run()

        print_delivery_schedule(optimal_route, min_distance)
```

```
=======================================================
        CUCKOO SEARCH OPTIMIZED ROUTE TIMELINE
=======================================================
Stop Order   | Action / Destination Location
-------------------------------------------------------
Stop 1       | Depart/Arrive at Starting Depot
Stop 2       | Depart/Arrive at City 1
Stop 3       | Depart/Arrive at City 2
Stop 4       | Depart/Arrive at City 3
Stop 5       | Depart/Arrive at City 4
Stop 6       | Return safely back to Starting Depot
=======================================================
Visual Route Map:
Depot -> City 1 -> City 2 -> City 3 -> City 4 -> Depot

Minimized Total Travel Distance: 24.05 miles/km
=======================================================
```